

שאלה 1

א. כתבו פונקציה שמקבלת מחרוזת ותו ומסירה מהמחרוזת את כל המופעים של התו. אין להקצות מערך נוסף (בצורה דינאמית או סטטית) לצורך הפעולה.

למשל: עבור `s="abracadabra"` ו-`ch='r'` הפונקציה תשנה את `s` להיות `s="abacadaba"` חתימת הפונקציה:

```
void remove_char(char s[], char ch)
```

ב. כתבו פונקציה שמקבלת שני מערכי `int` ואת האורכים שלהם ומחזירה מערך חדש שבו יהיו כל המספרים במערך הראשון שאין להם מחלק במערך השני. לצורך מימוש הפונקציה אין להשתמש באופרטור `[]`, כלומר מותר להשתמש אך ורק במצביעים. למשל: עבור המערכים

```
int s1[] = {1, 10, 5, 4, 21, 49, 24}, s2[] = {2, 3};
```

הפונקציה תחזיר `{1, 5, 49}` (הסדר לא משנה).

```
Int* divided_by(int* s1, unsigned int n1, int* s2, unsigned int n2)
```

שאלה 2

תור (Queue) הוא מבנה נתונים שבו:

- האיברים בתור נשמרים לפי סדר הכנסתם. אותו איבר יכול להופיע מספר לא מוגבל של פעמים
 - הכנסה של איבר חדש מתבצעת בהתחלת התור (פעולת ADD)
 - הוצאת איבר מתבצעת מסוף התור (פעולת REMOVE)
- למשל: אם נבצע את רצף הפעולות הבא:

```
ADD 5  
ADD 3  
ADD 5  
ADD 4
```

התור יראה כך (משמאל לימין): 4, 5, 3, 5

אם נבצע עכשיו פעולת REMOVE התור יכיל את האיברים 4, 5, 3

א. כתבו ממשק לתור גנרי שיכיל את הפעולות הבאות:

- QueueCreate – יצירת תור
- QueueDestroy – הריסת תור
- QueueAdd – הוספת איבר לתור. הפונקציה תקבל כתובת של איבר להכנסה ותכניס את האיבר שהועבר ללא יצירת עותק חדש.
- QueueRemove – הוצאת איבר מתור. אם התור ריק הפונקציה תחזיר NULL
- QueueSize – מספר האיברים בתור

ב. ממשו את:

- ה-`stdruuct` של התור. התור ימומש כרשימה מקושרת
- את הפונקציה `QueueDestroy`
- את הפונקציה `QueueRemove`

שאלה 3

בשאלה זו ניצור קידוד למחרוזות שמכילים רק ספרות (0-9), רווחים, תווי ירידת שורה ותו סיום מחרוזת (\0). במחרוזות רגילות נשמר כל תו בבייט, אולם במקרה שלנו ישנם רק 13 תווים אפשריים ולכן ניתן להסתפק בחצי בייט (4 ביטים) על מנת לשמור כל תו.

א. כתבו פונקציה שמקבלת מחרוזת כמתואר למעלה ומחזירה מערך שבו כל תו נשמר בחצי בייט. התווים 0-9 יקודדו על ידי המספרים 1-10 ('0'=1, '1'=2, וכו'), תו ירידת השורה יקודד על ידי 11 ואת תו הרווח נקודד על ידי 12. נשתמש ב-0 על מנת לציין את סיום המחרוזת. בסעיף זה מותר להשתמש רק בפעולות של ביטים על מנת ליצור את המערך (ולא במודולו, כפל או חילוק).

למשל: המחרוזת "012" תקודד בשני בטים:

- 1 יוכנס ל-4 הביטים התחתונים של הבית הראשון
- 2 יוכנס ל-4 הביטים העליונים של הבית הראשון
- 3 יוכנס ל-4 הביטים התחתונים של הבית השני
- 0 יוכנס ל-4 הביטים העליונים של הבית השני

המחרוזת "01" תקודד בשני בטים:

- 1 יוכנס ל-4 הביטים התחתונים של הבית הראשון
- 2 יוכנס ל-4 הביטים העליונים של הבית הראשון
- 0 יוכנס ל-4 הביטים התחתונים של הבית השני
- 4 הביטים העליונים של הבית השני הם לא חלק מהמחרוזת המכוצצת ("זבל").

חתימת הפונקציה:

```
char* compress(const char* s)
```

ב. כתבו פונקציה שמחשבת את אורך המחרוזת המכוצצת (בדומה ל-strlen). לצורך מימוש הפונקציה אין להשתמש בפעולות של ביטים.

חתימת הפונקציה:

```
int compressed_strlen(const char* compressed_str)
```

שאלה 4

סעיף א'

כתבו קוד שמבצע את הפעולות הבאות (ייתכן שהקוד מורכב משתיים או יותר פקודות):
1. הדפסת סכום האיברים במערך הנתון כאן:

```
arr=(0 1 2 3 4)
```

2. בדיקה האם קיים קובץ בשם Mary.txt בתיקייה הנוכחית, הדפסת Exists אם כן או Missing אם לא

סעיפים 3-4 מתייחסים לקובץ Mary.txt שמכיל את 7 השורות הבאות:

Mary had a little lamb, little lamb,
little lamb, Mary had a little lamb
whose fleece was white as snow.
And everywhere that Mary went
Mary went, Mary went, everywhere
that Mary went
The lamb was sure to go.

3. הדפסת מספר השורות שבהן Mary היא אחת משתי המילים הראשונות

4. הדפסת מספר השורות שבהן מופיעה המילה Mary ללא המילה lamb

סעיף ב'

כתבו סקריפט אשר מקבל שם של קובץ כקלט ומדפיס את מספר השורות בו. לתסריט אסור לקרוא לפקודה wc. על הסקריפט לבדוק שהקובץ שהועבר אכן קיים ולצאת לאחר הדפסת הודעת שגיאה אם לא.